

# Tutorial de Pseudocódigo

Esta sección se ha diseñado con un sentido totalmente didáctico, para servir de apoyo a las clases de pseudocódigo o algoritmia que forman parte de los cursos de programación.

Aunque no existen reglas comunes para la escritura de los pseudocódigos, he recogido una notación estándar que se utiliza en la mayor parte de los libros de programación en español.

Las palabras básicas reservadas, es decir, aquellas que pueden ser traducidas a palabras de un lenguaje de programación se presentan en minúscula cursiva.

Si tienes comentarios, críticas o sugerencias, sobre este tutorial, por favor envíame un mensaje.

## Datos y Tipos de Datos

Las cosas se definen en la computadora mediante datos, los algoritmos que vamos a diseñar van operar sobre esos datos. A nivel de la máquina los datos se representan como una serie de bits (dígito 1 ó 0). Los tipos de datos que vamos a manejar a lo largo del tutorial son : numéricos y carácter (también llamados alfanuméricos), existen además, los datos de tipo lógicos que solo pueden tener uno de dos valores : verdadero o falso.

Los datos numéricos pueden ser de varias clases: enteros, enteros largos, de doble precisión, de simple precisión, de coma flotante, reales; cuya definición depende del lenguaje de programación utilizado.

Los datos de tipo carácter o alfanuméricos están compuestos por el conjunto finito y ordenado de caracteres que la computadora reconoce:

caracteres alfabéticos : A,B,C,.....Z ; a,b,c,.....

caracteres numéricos : 0,1,2, .....9 (que no son números)

caracteres especiales : +, /, \*, %, \$, #, !, <, >, ã, !, {}, ~, etc.

Una cadena o string es una sucesión de caracteres que se encuentran delimitados por comillas ( " " ). La longitud de la cadena es la cantidad de caracteres que la forma, incluyendo los espacios que son un carácter más. Así:

"Asunción, Paraguay" es una cadena de longitud 18

"Miércoles 7 de Marzo de 2001" es una cadena de longitud 28 (el 7 y el 2001 no son números)

"123456" es una cadena de longitud 6, no es el número 123.456 sobre "123456" no se puede realizar ninguna operación aritmética como sumar, restar, etc, ya que se trata de una cadena alfanumérica.

## Variables

Cuando representamos datos, numéricos o alfanuméricos, debemos darles un nombre. Una variable es un nombre que representa el valor de un dato.

En esencia, una variable es una zona o posición de memoria en la computadora donde se almacena información. En un pseudocódigo y también en un programa se pueden crear tantas variables como querramos. Así tenemos:

A = 50; Variable tipo numérica A cuyo valor es 50.

Ciudad = "Asunción"; Variable alfanumérica o de tipo carácter Ciudad, cuyo valor es "Asunción"

X = C + B; Variable numérica X cuyo valor es la suma de los valores de las variables numéricas C y B. Es una variable calculada

Ten en cuenta que las operaciones que se pueden realizar con dos o más variables exigen que éstas sean del mismo tipo. No podemos "sumar", por ejemplo una variable alfanumérica a otra numérica y viceversa como por ejemplo:

FechaNueva="1 de Junio de 1.971" + 5

### **Esto no se puede hacer !!**

Para dar nombres a las variables hay que seguir ciertas reglas:

Pueden tener hasta 40 caracteres

Debe empezar obligatoriamente con una letra  
(a-z o A-Z)

No pueden contener espacios en blanco

El resto de los dígitos pueden ser números

Se pueden incluir caracteres especiales como el guión o el punto.

### **Ejemplos de nombres válidos de variables**

FechaNueva  
C1  
totalGuaranies  
CONTADOR-5  
H123  
cantidad\_de\_Alumnos  
Pedido.Almacen

### **Ejemplos de nombres de variables NO válidos**

Fecha nueva  
1contador  
24ABC  
primer-valor N

Algunos lenguajes de programación exigen la declaración de las variables que se van a utilizar en todo el programa; es decir, que al comenzar el programa se debe decir que nombre tiene, de que tipo es (numérica o alfanumérica) y un valor inicial. Como aquí no estamos tratando con ningún lenguaje, la declaración de las variables puede omitirse.

Las variables también pueden inicializarse; darles un valor inicial. Por defecto, todas las variables para las que no especifiquemos un valor inicial, valen cero si son de tipo numérica y nulo (nulo no es cero ni espacio en blanco; es nulo) si son de tipo carácter.

### **Operaciones**

Las variables se pueden procesar utilizando operaciones apropiadas para su tipo.

Los operadores son de 4 clases:

Relacionales  
Aritméticos  
Alfanuméricos  
Lógicos

Los operadores relacionales se utilizan para formar expresiones que al ser evaluadas producen un valor de tipo lógico: verdadero o falso. Ellos son:

| <b>Signo</b> | <b>Operador</b>   |
|--------------|-------------------|
| >            | Mayor que         |
| <            | Menor que         |
| =            | Igual a           |
| <=           | Menor o igual que |
| >=           | Mayor o igual que |
| <>           | Distinto          |

Ejemplos:

| <b>Ejemplo</b> | <b>Resultado</b> |
|----------------|------------------|
| 25 <= 25       | Verdadero        |
| 25 <> 25       | Falso            |
| 25 <> 4        | Verdadero        |
| 50 <= 100      | Verdadero        |
| 500 >= 1       | Verdadero        |
| 1 = 6          | Falso            |

Cuando se comparan caracteres alfanuméricos se hace uno a uno, comenzando por la izquierda hacia la derecha.

Si las variables son de diferente longitud, pero exactamente iguales, se considera que la de menor longitud es menor.

Los datos alfanuméricos son iguales si y solo si tienen la misma longitud y los mismos componentes.

Las letras minúsculas son mayores que las mayúsculas y cualquier carácter numérico es menor que cualquier letra mayúscula o minúscula; Así:

carácter numérico < mayúsculas < minúsculas.

Ejemplos:

| <b>Comparación</b> | <b>Resultado</b> |
|--------------------|------------------|
| "A" < "B"          | Verdadero        |
| "AAAA" > "AAA"     | Verdadero        |
| "B" > "AAAA"       | Verdadero        |
| "C" < "c"          | Verdadero        |
| "2" < "12"         | Falso            |

Estas comparaciones se realizan utilizando el valor ASCII de cada carácter. Para tratar los números se utilizan los operadores aritméticos:

| <b>Signo</b> | <b>Significado</b>          |
|--------------|-----------------------------|
| +            | Suma                        |
| -            | Resta                       |
| *            | Multiplicación              |
| /            | División                    |
| ^            | Potenciación                |
| MOD          | Resto de la división entera |

El único operador alfanumérico se utiliza para unir o concatenar datos de este tipo:

| <b>Signo</b> | <b>Significado</b> |
|--------------|--------------------|
| +            | Concatenación      |

Ejemplos:

| <b>Expresión</b>      | <b>Resultado</b> |
|-----------------------|------------------|
| "Pseudo" + "código"   | "Pseudocódigo"   |
| "3" + "4567"          | "34567"          |
| "Hola " + "que tal ?" | "Hola que tal ?" |

Los operadores lógicos combinan sus operandos de acuerdo con las reglas del álgebra de Boole para producir un nuevo valor que se convierte en el valor de la expresión, puede ser verdadero o falso.

| Signo | Significado         |
|-------|---------------------|
| OR    | Suma lógica (O)     |
| AND   | Producto lógico (Y) |
| NOT   | Negación (NO)       |

Ejemplos:

| Expresión        | Resultado |
|------------------|-----------|
| Verdad AND Falso | Falso     |
| NOT Falso        | Verdad    |
| Verdad OR Falso  | Verdad    |

Por ejemplo, la expresión:  $(12 + 5) \text{ OR } (7 + 3) = 10$  es verdadera (se cumple una y verdad OR Falso es Verdad).

La expresión  $(12 * 5) \text{ AND } (3 + 2) = 60$  es falsa (verdad AND falso = Falso).

¿Cómo se evalúan los operadores? La prioridad de los operadores es:

1. Paréntesis
2. Potencias
3. Productos y Divisiones
4. Sumas y restas
5. Concatenación
6. Relacionales
7. Lógicos

## ASIGNACIONES

La operación de dar valor a una variable se llama asignación. La asignación vamos a representarla con el símbolo  $\leftarrow$ ; una flecha apuntando a la izquierda. No utilizaremos en signo = ya que el operador de asignación varía de acuerdo con el lenguaje de programación utilizado. El formato general de una asignación es:

***nombre de la variable  $\leftarrow$  expresión***

La flecha se sustituye en los lenguajes de programación por = (basic); := (pascal). Pero para evitar ambigüedades en el pseudocódigo utilizaremos la flecha para la asignación y el símbolo = **para** indicar igualdad. He aquí algunos ejemplos:

$A \leftarrow 100$ ; significa que a la variable A se le ha asignado el valor 100, ahora A vale 100.  
 $\text{suma} \leftarrow 5+10$ ; asigna el valor 15 a la variable suma a través de una asignación aritmética.  
 $x \leftarrow z + v$ ; asigna el valor de la suma de las variables z y v a la variable x. El resultado depende de los valores que se asignen a x y a z

**Toda asignación es destructiva.** Esto quiere decir que el valor que tuviera antes la variable se pierde y se reemplaza por el nuevo valor que asignamos, así cuando se ejecuta esta secuencia:

$B \leftarrow 25$   
 $B \leftarrow 100$   
 $B \leftarrow 77$

el valor final que toma B será 77 pues los valores 25 y 100 han sido destruidos.

Cuando una variable aparece a ambos lados del símbolo de asignación como:

$C \leftarrow C + 1$

conviene inicializarlas al comenzar el programa con cero, aunque no es obligatorio por ahora (en algunos lenguajes de programación sí es necesario).

Recordemos que no se pueden asignar valores a una variable de un tipo diferente al suyo.

Pongamos atención a este ejemplo de asignaciones:

$A \leftarrow A + 2 * B$

$B \leftarrow C - A$

En las dos primeras acciones, A toma el valor 3 y B el valor 4.

$C \leftarrow A + 2 * B$

La expresión tomará el valor  $3 + 2 * 4 = 3 + 8 = 11$

C vale entonces 11.

$B \leftarrow C - A$

C vale 11, A vale 3, por lo tanto B valdrá  $11 - 3 = 8$

Como toda asignación es destructiva, el valor anterior de B se pierde y pasa a valer ahora 8.

Otro ejemplo:

$J \leftarrow J * 3$

Que valor tiene J al final ? Veamos.

Primero se asigna 33 a la variable J, J vale entonces 33; luego:

$J \leftarrow J + 5$

Esto es: Sumar 5 al valor de J y asignarlo a la variable J. J vale 33.

$J \leftarrow 33 + 5$ ;  $J \leftarrow 38$

J vale ahora 38.

El valor anterior que era 33 se destruyó.

Seguimos:  $J \leftarrow J * 3$

Esto es: Multiplicar 3 al valor de J y asignarlo a la variable J.

$J \leftarrow 38 * 3$ ;  $J \leftarrow 114$

El valor final de J es 114.

### **Entrada y Salida de Información**

Los datos que vamos a obtener del usuario para procesarlos también deben ser asignados a variables, la operación de lectura, lee un dato y lo asigna a una variable. La instrucción para la lectura de un dato es *leer* o también *ingresar*. Por ejemplo:

***leer numero***

Esta instrucción pide al usuario un valor que será asignado a la variable numero, es decir, en numero se almacena el valor ingresado por el usuario

***leer Edad, Peso, Sexo***

Representa la lectura de tres valores que se van a almacenar en las variables Edad, Peso y Sexo; en ese mismo orden.

Ya tenemos nuestro primer comando: *leer*

Ahora bien, cuando queramos mostrar el resultado del algoritmo, un mensaje, un valor, etc, vamos a utilizar el comando *imprimir*. Por ejemplo:

*imprimir* "Hola" ; muestra en la pantalla el mensaje Hola, Hola va entre comillas porque es una cadena.

*imprimir* A; muestra en la pantalla el valor que está almacenado en la variable A.

*imprimir* "El valor del promedio es:", promedio

Esta instrucción muestra el mensaje que está entre comillas y luego el valor de la variable

promedio. La coma separa el mensaje de la variable. Si promedio vale 5, lo que se verá en la pantalla será:

El valor del promedio es: 5

Ya conocemos dos comandos que vamos a utilizar en nuestros pseudocódigos: *leer* e *imprimir*

También podemos mostrar un mensaje cuando solicitamos algún dato al usuario por medio del comando leer así: *leer* "Ingrese su edad", edad

El valor de la variable que pedimos al usuario se asigna a edad. Esta instrucción se verá así en la pantalla: Ingrese su edad ?

El símbolo de interrogación aparece automáticamente cada vez que usamos el comando *leer*.

Entonces, en la escritura de pseudocódigos, las acciones de lectura y escritura se representan por los siguientes formatos:

***leer* Variable o lista de variables separadas por comas.** Ejemplos:

*leer* Edad

*leer* Ciudad, Pais

***imprimir* Variable o lista de variables separadas por comas.**

Ejemplos:

*imprimir* promedio

*imprimir* TotalMes, TotalAño, TotalGeneral

*imprimir* "Así se muestra un mensaje o comentario"

En resumen: Las instrucciones disponibles para escribir un programa dependen del lenguaje de programación utilizado. Existen instrucciones -o acciones- básicas que se pueden implementar de modo general en cualquier algoritmo y que soportan todos los lenguajes de programación. Estas son:

- 1- Instrucciones de inicio/fin
- 2- Instrucciones de asignación
- 3- Instrucciones de lectura
- 4- Instrucciones de escritura

| Tipo de Instrucción | Pseudocódigo               |
|---------------------|----------------------------|
| Comienzo de proceso | <i>inicio</i>              |
| Fin de proceso      | <i>fin</i>                 |
| Entrada (Lectura)   | <i>leer</i>                |
| Salida (Escritura)  | <i>imprimir o escribir</i> |
| Asignación          | <i>←</i>                   |

---

### Resolución de Problemas

Antes de resolver un problema por medio de un pseudocódigo, es necesario definirlo y comprenderlo claramente. Leeremos con atención el enunciado del problema y una vez comprendido responderemos a las preguntas:

¿ Qué información debe proporcionar la resolución del problema?

¿ Cuáles son los datos que necesito para resolver el problema?

La respuesta de la primera pregunta nos dice que salidas va a proporcionar el algoritmo y la segunda qué datos se nos proporcionan para resolver el problema y cuáles debemos calcularlos.

Problema: **Leer las longitudes de un rectángulo y calcular la superficie y el perímetro.**

Para calcular el área y el perímetro de un rectángulo, se necesitan las medidas del ancho y el alto, estas medidas serán leídas en dos variables.

Las salidas serán los valores del área y el perímetro que serán calculados utilizando fórmulas.

Entradas: largo, ancho

Salidas: perímetro, área  
El pseudocódigo es:

```
inicio
    leer largo
    leer ancho
    perimetro  $\leftarrow$  largo + ancho * 2
    area  $\leftarrow$  largo * ancho
    imprimir perimetro
    imprimir area
fin
```

Problema: **Escribir un pseudocódigo que intercambie el valor de dos variables.**

Si se tienen, por ejemplo  $A = 5$  y  $B = 10$ , se quiere intercambiar el valor de las variables, así:  
 $A = 10$ ;  $B = 5$ .

No podemos asignar directamente el valor de una a la otra porque uno de los valores se destruiría; de modo que esto no se puede hacer:

$A \leftarrow B$  (el valor de A se pierde y quedaría  $A = 10$  ;  $B = 10$ )

La solución consiste en asignar el valor de una de las variables a otra variable auxiliar.

```
inicio
leer A, B
A  $\leftarrow$  B
B  $\leftarrow$  Auxiliar
imprimir A, B
fin
```

Sigamos paso a paso el pseudocódigo:

*leer A, B* ..... Se pide al usuario dos valores. Supongamos que se ha ingresado  $A = 100$  ;  $B = 5$   
*Auxiliar  $\leftarrow$  A* ..... Se asigna a Auxiliar el valor 100. Auxiliar vale 100.  
El valor de las variables es:

|                 |
|-----------------|
| <b>A</b>        |
| <b>B</b>        |
| <b>Auxiliar</b> |
| 100             |
| 5               |
| 100             |

$A \leftarrow B$  ..... Se asigna a A el valor de B para intercambiar. Ahora el valor de las variables es:

|                 |
|-----------------|
| <b>A</b>        |
| <b>B</b>        |
| <b>Auxiliar</b> |
| 5               |
| 5               |
| 100             |

$B \leftarrow$  Auxiliar ..... El valor de A que se guardó en Auxiliar se asigna a B para el intercambio.

|                 |
|-----------------|
| <b>A</b>        |
| <b>B</b>        |
| <b>Auxiliar</b> |
| 5               |
| 100             |
| 100             |

El intercambio está hecho. Luego se imprimen los respectivos valores ya intercambiados con la línea:

*imprimir A, B*

## Contador

Un contador es una variable destinada a contener valores que se van incrementando o decrementando cada vez que se ejecuta la acción que lo contiene. El incremento o decremento es llamado paso de contador y es siempre constante.

Por ejemplo; el marcador de un partido de fútbol, cada vez que un equipo anota un gol, aumenta su marcador en una unidad.

En las carrera de automóviles, cada vez que un vehículo pasa por la línea de meta, se incrementa en una unidad el número de vueltas dadas al circuito, o bien se decrementa en una unidad el número de vueltas que quedan por realizar.

Aunque el incremento es siempre constante, el paso de contador no necesariamente puede ser la unidad como en los ejemplos que se han dado más arriba; también puede incrementarse o decrementarse a de dos, tres, cuatro,... n; es decir, puede ser cualquier número que conserva el mismo valor durante todo el programa.

La sintaxis de una variable contador es:  $C \leftarrow C + 1$  o  $C \leftarrow C - 1$

variable  $\leftarrow$  variable + constante (al incrementar)

variable  $\leftarrow$  variable - constante (al decrementar)

Ejemplos:

gol\_local  $\leftarrow$  gol\_local + 1

vueltas  $\leftarrow$  vueltas + 1

faltan  $\leftarrow$  faltan - 1

de\_cinco  $\leftarrow$  de\_cinco + 5

c  $\leftarrow$  c + 1

x  $\leftarrow$  x - 3

Observación: Cuando una variable aparece a ambos lados del símbolo de asignación, conviene inicializarlas a cero.

## Acumulador o Sumador

Es una variable que nos permite guardar un valor que se incrementa o decrementa en forma NO constante durante el proceso. En un momento determinado tendrá un valor y al siguiente tendrá otro valor igual o distinto. Por ejemplo; cuando realizamos un depósito en el banco, la cantidad depositada cada vez no es siempre la misma; unas veces será una cantidad y otras veces distinta. Lo mismo ocurre cuando realizamos algún retiro, pero decrementando la cantidad total.

La sintaxis es:

acumulador  $\leftarrow$  acumulador + variable (al incrementar)

acumulador  $\leftarrow$  acumulador - variable (al decrementar)

acumulador es la variable en la que se almacena el resultado.

variable contiene el número que estamos incrementando o decrementando

Ejemplos:

saldo  $\leftarrow$  saldo + entrega

saldo  $\leftarrow$  saldo - retiro

suma  $\leftarrow$  suma + numero

A  $\leftarrow$  A + edad



# ESTRUCTURA DE SELECCIÓN

La estructura de selección, se ejecuta condicionalmente, es decir, si una cierta condición es verdadera se ejecuta un bloque de instrucciones, si es falsa se ejecuta un bloque diferente de instrucciones. Por ejemplo, si en el cine proyectan "Star Wars Episode I", entonces hay que formar fila para comprar los billetes e ingresar al cine, si no, decidimos otra actividad como ir a bailar.

Si utilizamos una selección es para indicar que según el resultado cierto o falso de una expresión vamos a tomar una decisión de realizar determinadas acciones especificadas; seleccionamos las acciones a realizar.

La instrucción que permite tomar una decisión, basada en una condición es **Si...Entonces**. Al evaluar la condición, **Si...entonces** puede devolver solo dos resultados posibles: Verdadero o Falso; es decir, Si o No. El formato de la estructura de selección es:

**si <condición> entonces**

instrucción 1  
instrucción 2  
.....  
instrucción n

**si-no**

instrucción a  
instrucción b  
.....  
instrucción z

**fin-si**

Observa como el sangrado permite identificar fácilmente que grupo de instrucciones se ejecutan en cada caso.

Por ejemplo, Cuando realizamos una llamada telefónica:

**Si {señal de ocupado} entonces**

Colgar el teléfono

**si - no**

Iniciar la conversación

**fin - si**

En este caso, la condición es {señal de ocupado}, que puede ser verdadera o falsa. Si es verdadera, entonces debemos colgar el teléfono y si no, podemos relizar la conversación.

Ejemplo:

**Si A = 5 entonces**

imprimir "A es 5"

**si - no**

imprimir "A no es igual a 5"

**fin - si**

*Tambien puede obviarse el si - no cuando no nos interesa ejecutar un bloque de instrucciones en caso de que la condición no se cumpla.*

**Si {condición} entonces**

instrucción 1

instrucción 2

.....

instrucción n

**fin - si**

Por ejemplo;

**Si {hoy es Miércoles} entonces**  
**Comprar entradas para el cine**  
**fin - si**

### Ejemplos

**Introducir un número por teclado y determinar si es positivo o negativo.-**

Para saber si un número es positivo o negativo, debemos saber si es menor o mayor a cero. Si es mayor, el número es positivo y si es menor resulta negativo. Utilizamos **Si...** para evaluar como es el número con respecto a cero y mostramos los mensajes correspondientes en cada caso. Así:

**inicio**

**leer Numero**

**Si Numero < 0 entonces**

**imprimir "El número es negativo"**

**si-no**

**imprimir "El número es positivo"**

**fin-si**

**fin**

Ejemplo 2. **Dados dos números, establecer cuál es mayor .**

Comenzamos leyendo ambos números, que en el ejemplo se llamarán NumeroA y NumeroB. Luego debemos comparar como es uno contra el otro (puede ser NumeroA contra NumeroB o bien comparar NumeroB contra NumeroA):

**inicio**

**leer NumeroA, NumeroB**

**Si NumeroA < NumeroB entonces**

**imprimir "El mayor es:", NumeroB**

**si-no**

**imprimir "El mayor es:", NumeroA**

**fin-si**

**fin**

En este ejemplo, que pasaría si los números fueran iguales?. Hagamos la prueba

Luego de leer los números, por ejemplo: NumeroA=100 y NumeroB=100 se ejecutan las instrucciones:

*Si NumeroA < NumeroB entonces*

*imprimir "El mayor es:", NumeroB*

El resultado de la condición

Por lo tanto, al ser falsa la condición, se ejecuta la instrucción *imprimir "El mayor es:", NumeroA*.

Por tanto, el algoritmo ofrecerá una solución incorrecta cuando los números son iguales. Para solucionar esto, tenemos que prever el caso de que los números sean iguales.

*inicio*

**leer NumeroA, NumeroB**

**Si NumeroA < NumeroB entonces**

**imprimir "El mayor es:", NumeroB**

**si-no**

**Si NumeroA > NumeroB entonces**

**imprimir "El mayor es:", NumeroA**

**si-no**

**imprimir "Los números son iguales"**

**fin-si**

**fin-si**

**fin**

Esta solución contiene dos estructuras de repetición, una dentro de la otra (anidada). En caso de ser necesario podemos anidar tantas estructuras de selección como sea necesario. El algoritmo averigua si A es menor a B, si no lo es, tenemos otras dos posibilidades: que sea menor o igual, esto es lo que determina la estructura anidada.

Otro ejemplo de estructuras de repetición anidadas, consiste en dado un número del 1 al 7, establecer al día de la semana.

*inicio*

*leer numero*

```

Si numero=1 entonces
    imprimir "Domingo"
si-no
    Si numero=2 entonces
        imprimir "Lunes"
    si-no
        Si numero=3
            imprimir "Martes"
        si-no
            Si numero=4 entonces
                imprimir "Miércoles"
            si-no
                Si Numero=5 entonces
                    imprimir "Jueves"
                si-no
                    Si numero=6 entonces
                        imprimir "Viernes"
                    si-no
                        Si numero=7 entonces
                            imprimir "Sábado"
                        si-no
                            imprimir "El número debe estar entre 1 y 7"
                        fin-si
                    fin-si
                fin-si
            fin-si
        fin-si
    fin-si
fin

```

Notarás que tenemos varios *Si...entonces* anidados, ya que si el número ingreso no es 1, tenemos que preguntar si es 2 ó 3 ó 4...etc. El último *Si...entonces* es para verificar que el número ingresado no es 1, 2, 3, 4, 5, 6 ó 7; sino cualquier otro que no nos interesa.

Repasa los algoritmos anteriores.

Resulta bastante tedioso anidar un montón de Si ... entonces, como en el ejemplo del día de la semana. Cuando queramos o necesitemos hacer numerosas comparaciones podemos usar otra estructura de selección llamada *Según Sea*. El formato de estructura de selección *Según sea* es:

```

Según sea <variable>
    Caso = <condición>
    Caso = <condición>
        instrucción o instrucciones
    Otro caso
        instrucción o instrucciones
fin-según

```

Así, utilizando esta estructura, el problema del día de la semana será así:

```

inicio
    Leer numero
    Según sea numero
        Caso = 1
            imprimir "Domingo"
        Caso = 2
            imprimir "Lunes"
        Caso = 3
            imprimir "Martes"
        Caso = 4
            imprimir "Miércoles"
        Caso = 5
            imprimir "Jueves"
        Caso = 6
            imprimir "Viernes"

```

```

Caso = 7
    imprimir "Sábado"
Otro Caso
    imprimir "El número debe estar entre 1 y 7"
fin-según
fin

```

Lo cual resulta menos engorroso que varios *Si... entonces* anidados. Es posible anidar *Si... entonces* dentro de estructuras *Según sea* y viceversa.

Observa que la instrucción *Otro Caso* ejecuta una o varias instrucciones cuando no se cumple ningún caso de los contemplados más arriba. *Otro Caso* debe estar siempre al final (cuando sea necesario, si no o es se puede omitir *Otro Caso*)

El uso de una u otra estructura depende de cada persona, pero en general cuando las condiciones a evaluar son muchas, se utiliza *Según Sea*.-

La estructura según sea admite varias condiciones por ejemplo:

```

Según sea MES
    caso= 1,3,5,7,8,10,12
        TDías ≤ 31
    caso = 2,4,6,11
        TDías ≤ 30
    caso = 2
        TDías ≤ 28
fin-según

```

Este pequeño ejemplo establece el número de días de un mes determinado almacenado en la variable MES (para años no bisiestos). En lugar de escribir varios *Caso= 1*, *Caso =2*, etc, se puede especificar acción o acciones (en este caso la asignación de días a *TDías*) cuando la variable tome uno de los valores separados por comas. Es decir si *TDías* es 1 ó 3 ó 5 ó 7 ó 8 ó 10 ó 12; se ejecuta *TDías=31*.

## REPETICION

La estructura repetitiva se utiliza cuando se quiere que un conjunto de instrucciones se ejecuten un cierto número finito de veces. Llamamos bucle o ciclo a todo proceso que se repite un cierto número de veces dentro de un pseudocódigo o un programa.

Existen dos tipos de estructuras repetitivas; la primera es aquella en donde se tiene perfectamente establecido el número de veces que un grupo de acciones se van a ejecutar (20, 5, 2 veces), y la segunda en la que el número de repeticiones es desconocido y se hará hasta que se cumpla o no cierta condición.

Un ejemplo de la primera sería imprimir los datos de los alumnos de una clase (se conoce cuantos alumnos hay) y un ejemplo de la segunda puede ser el mostrar un mensaje de error cada vez que el usuario pulse una determinada tecla (no sabemos cuantas veces pulsará esa tecla). Las acciones que forman parte del cuerpo del bucle son ejecutadas de forma repetitiva mediante la ocurrencia o no de una condición.

Cuando conocemos de antemano el número de veces en que se desea ejecutar una acción o grupo de acciones, se utiliza la estructura repetitiva Desde o Para.

Esta estructura ejecuta las acciones del cuerpo del bucle un número especificado de veces, y de modo automático controla el número de iteraciones o pasos.

La sintaxis es:

**Desde variable**  $\leq$  **Vi hasta Vf [incremento]**  
**acción o acciones**

**fin-desde**

Donde:

variable: variable índice

Vi: valor inicial de la variable índice

Vf: valor final de la variable índice

[incremento]: el número que se incrementa (o decrementa) a la variable índice en cada iteración del bucle, si se omite es 1.

Ejemplo:

Imprimir todos los números del 1 al 100.

**Inicio**

**desde I ↗ 1 hasta 100**

**imprimir I**

**fin-desde**

**fin**

I es la variable índice con un valor inicial de 1, se incrementa uno en cada paso hasta 100.

Podemos notar que la estructura desde comienza con un valor inicial de la variable índice y las acciones se ejecutan hasta que el valor inicial sea MAYOR que el que el Valor final.

La variable índice se incrementa en uno (en el ejemplo) y si este nuevo valor del índice no es mayor que el valor final, se ejecuta de nuevo la acción imprimir.

En este caso se visualizará los números 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 ....97, 98, 99, 100 El incremento o paso de contador es siempre 1 si no es especifica lo contrario.

Otro Ejemplo: *Imprimir todos los números pares desde 2 hasta el 300*

**Desde I ↗ 2 hasta 300 incremento 2**

**imprimir I**

**fin-desde**

Donde:

La variable índice comienza en 2, se imprime 2 y en el siguiente paso se incrementa (suma) 2 a la variable índice que pasa a valer 4; se imprime el 4 y como 4 es menor que 300 (valor final) , se pasa a una nueva iteración incrementando nuevamente en 2 el índice, que ahora vale 6; y así sucesivamente...

Aquí se visualizan: 2, 4, 6, 8, 10, 12, 14, 16, 18, ..... 296, 298, 300

Si deseamos mostrar los *impares* el algoritmo es el siguiente:

**Desde I ↗ 1 hasta 300 incremento 2**

**imprimir I**

**fin-desde**

La variable índice toma estos valores:

| Indice o Paso (veces que se ejecuta el ciclo) | Valor de I |
|-----------------------------------------------|------------|
| 1                                             | 1          |
| 2                                             | 3          |
| 3                                             | 5          |
| 4                                             | 7          |
| 5                                             | 9          |
| 6                                             | 11         |
| ....                                          | ....       |
| 150                                           | 299        |

Vemos los valores: 1, 3, 5, 7, 9, 11, 13, .... , 299

El ciclo termina mostrando 299 puesto que en el siguiente paso, La variable I valdría 301 y es mayor al límite establecido de 300. I pasa a valer realmente 301 en el último paso, solo que la instrucción de imprimir no se ejecuta porque el límite de 300 se supera.

Si diéramos la instrucción de imprimir el valor de I, inmediatamente después del fin-desde, veíamos 301.

Ejemplo 3: *Imprimir los valores comprendidos entre el 460 y 500 en orden inverso.*

Debemos mostrar: 500, 499, 498, 497, ..... 462, 461, 460. En este caso haremos un decremento a la variable índice (no un incremento como en los ejemplos anteriores). Tenemos que comenzar nuestra variable índice en 500 y decrementar una unidad hasta alcanzar el 460, así:

**Desde I 500 hasta 460 incremento -1**

**imprimir I**

**fin-desde**

| Indice o Paso (veces que se ejecuta el ciclo) | Valor de I |
|-----------------------------------------------|------------|
| 1                                             | 500        |
| 2                                             | 499        |
| 3                                             | 498        |
| 4                                             | 497        |
| 5                                             | 496        |
| ....                                          | ....       |
| 39                                            | 462        |
| 40                                            | 461        |
| 41                                            | 460        |

Como salida tenemos, entonces: 500, 499, 498, 497, 496, 495, 494, .... 464, 463, 462, 461, 460.

El segundo tipo de estructura repetitiva se diferencia de la primera en que no se conoce el número de repeticiones o iteraciones en que se va a ejecutar una instrucción o un bloque de instrucciones.

Estas estructuras son básicamente dos: Estructura **mientras....fin-mientras** y la estructura **repetir....hasta**. Estas dos se diferencian en que la verificación de la condición para repetir el ciclo se hace al inicio con *mientras* y al final con *repetir*

También existen estructuras repetitivas que son combinaciones de estas dos que mencionamos, pero aquí no las estudiaremos.

### Estructura Mientras

Como su nombre lo indica, esta estructura repite el cuerpo del bucle **mientras** se cumpla una determinada condición. Su sintaxis es:

**mientras {condición}**

**acción 1**

**acción 2**

**acción 3**

**.....**

**acción n**

**fin mientras**

**instrucción X**

Lo primero que el computador hace es examinar la condición, lo que puede dar como resultado dos posibilidades:

- La condición se cumple: Se ejecutan acción 1, acción 2, acción 3, ..., acción n.

Las estará repitiendo hasta que la condición no se cumpla, entonces se sale del ciclo y se siguen ejecutando la o las instrucciones que vienen a continuación y están fuera del bucle; instrucción X.

- La condición no se cumple: No entrará en el ciclo. Se ejecutan las instrucciones que vienen después del bucle, instrucción X, por ejemplo.

De esto se deduce que el cuerpo del bucle de una estructura mientras puede repetirse **cero o más veces**, veces que son determinadas por el cumplimiento o no de la condición.

#### Ejemplo

**mientras contraseña < > "josua"**

**imprimir "La contraseña es incorrecta !"**

**fin-mientras**

**imprimir "Ha ingresado la contraseña correcta"**

Veremos más ejemplos de esta estructura en la sección ejercicios. Al analizarlos comprenderemos mejor como funciona. -

#### Estructura Repetir

La estructura repetir cumple la misma función que la estructura mientras. La diferencia está en que la estructura mientras comprueba la condición al inicio y repetir lo hace al final; por eso la estructura repetir se ejecuta **por lo menos una vez**.

La sintaxis es:

**repetir**

**instrucción 1**

**instrucción 2**

**instrucción 3**

**.....**

**hasta {condición}**

**instrucción X**

Repetir es opuesta a la estructura mientras. Repetir se ejecuta hasta que se cumpla una condición que se comprueba al final del bucle. Esto implica que las instrucciones que forman el cuerpo del bucle se ejecutan por lo menos una vez. Con la estructura mientras el bucle puede ejecutarse 0 o más veces.

Lo que la computadora hace al ejecutar la estructura repetir es:

- Se ejecutan: instrucción 1, instrucción 2, instrucción 3, .....

- Se evalúa la condición. Si esta es **FALSA** se vuelve a repetir el ciclo y se ejecutan instrucción 1, instrucción 2, instrucción 3, .....

Si la condición es **VERDADERA** se sale del ciclo y se ejecuta instrucción X.

Recordemos una vez más las diferencias entre las estructuras mientras y repetir

| MIENTRAS                                                                                            | REPETIR                                                                            |
|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Comprobación de la condición al inicio, antes de entrar al bucle                                    | Comprobación de la condición al final, después de haber ingresado una vez al bucle |
| Las instrucciones del cuerpo del bucle se ejecutan en forma repetitiva si la condición es verdadera | Las instrucciones del cuerpo del bucle se ejecutan si la condición es falsa        |
| Las acciones del bucle se pueden ejecutar 0 o más veces                                             | Las acciones del bucle se ejecutan por lo menos una vez                            |

#### Ejemplo

**repetir**

**imprimir "La contraseña es incorrecta !"**

**hasta** contraseña = "josua"

Más ejemplos en la sección Ejercicios.

En resumen, hemos visto dos tipos de estructuras repetitivas, el primer tipo en la que conocemos el número de veces que se repetirá el bucle o ciclo (**Desde ....fin-desde**); y el segundo tipo en el cual no conocemos el número de veces en se repite el ciclo ya que está determinado por el cumplimiento o no de una condición (**mientras ..... fin-mientras** y **repetir....hasta**).

Toda estructura **Desde....fin-desde** tiene una estructura **mientras....fin-mientras** o **repetir.....hasta** equivalente. Sin embargo no toda estructura **mientras...** o **repetir ...** tiene un **Desde ....fin-desde** equivalente.

## VECTORES

Hasta ahora hemos trabajado con datos simples que representaban un número, un carácter o una cadena. Sin embargo, en ocasiones se necesita procesar una colección de valores que están relacionados entre sí por algún método, por ejemplo, una lista de calificaciones, de los meses del año, temperaturas a lo largo de una semana, etc.

El procesamiento de estos datos utilizando datos simples es muy difícil. Por eso, se han definido en la programación varias estructuras de datos, que son una colección caracterizada por alguna organización y por las operaciones que se definen en ella.

Una de estas estructuras son los vectores.

Un vector es un conjunto de elementos del mismo tipo que comparten un nombre común; algo así como una variable que puede almacenar al mismo tiempo más de un valor.

Los vectores reciben también el nombre de tablas, listas o *arrays*.

Un vector es un conjunto ordenado y homogéneo. Ordenado porque el primer elemento, segundo, tercero... n-ésimo puede ser identificado y homogéneo porque sus elementos son todos del mismo tipo (numéricos o alfanuméricos, pero no una combinación de ambos).

Gráficamente, un vector se representa como una tabla:

De igual forma que cualquier variable, un vector debe tener un nombre.

Aquí hemos llamado A a nuestro vector ejemplo.

Los elementos que están en el vector A ocupan todos una determinada posición dentro de él:

Así, el número -5 se encuentra en la posición 3; el 99 en la posición 10 y el 12 en la posición 1.

**A(3) = - 5**

**A(10) = 99**

**A(1) = 12**

Vemos, entonces que un elemento se referencia por el nombre del vector y la posición que ocupa dentro de él. El número que se coloca entre paréntesis se llama índice y designa la posición del elemento en el vector.

Cada elemento del vector se puede procesar como si fuera una variable simple.

La dimensión de un vector está dada por la cantidad de elementos que contiene y debe ser definida al comenzar el programa.

### Cargar un vector



La asignación de valores a los elementos de un vector se realiza de esta forma:

**A(4)  $\leftarrow$  0**

**A(7)  $\leftarrow$  4**

**etc.**

Por lo tanto, para que el vector A contenga los valores que se han mostrado como ejemplo, se han hecho estas asignaciones:

**A(1)  $\leftarrow$  12**

**A(2)  $\leftarrow$  50**

**A(3)  $\leftarrow$  -5**

**A(4)  $\leftarrow$  0**

**A(5)  $\leftarrow$  100**

**A(6)  $\leftarrow$  33**

**A(7)  $\leftarrow$  4**

**A(8)  $\leftarrow$  7**

**A(9)  $\leftarrow$  5**

**A(10)  $\leftarrow$  99**

Por supuesto que no vamos a cargar un vector de 100 elementos, escribiendo 100 asignaciones.

La carga de un vector se hace por medio del ciclo *desde....fin desde* . Nuestro bucle va a comenzar en 1 hasta un número N que es la longitud del vector.

Pero antes, no demos olvidar que el vector debe ser dimensionado. Al dimensionar un vector le decimos a la máquina que reserve los espacios de memoria necesarios para los elementos del vector.

Los problemas relacionados con vectores tendrán casi siempre esta forma

**inicio**

**Leer dimensión del vector**

**Dimensionar Vector**

**Cargar Vector**

**Procesamiento de los elementos del vector (según lo que se pida)**

**Imprimir Vector**

**fin**

Por lo tanto, vamos a tener tres *desde...fin desde* bien diferenciados:

1. Un bucle para la carga
2. Un bucle para el proceso
3. Un bucle para la impresión

Aunque la carga y el proceso pueden hacerse dentro de un solo bucle, particularmente no recomiendo esta práctica, ya que casi siempre crea dificultades innecesarias.

### Cargar el vector

Ejercicio: Cargar un vector de 30 componentes.

No olvidemos que antes de cargar un vector debemos dimensionarlo. En este caso la dimensión del vector es 30.

Luego habilitaremos un bucle *desde....fin desde* comenzando en 1 hasta 30. Llamaremos V a nuestro vector.

**inicio**

**dimensionar V(30)**

**desde I  $\leftarrow$  1 hasta 30**

**Leer V(I)**

**fin desde**

**fin**

De esta forma hemos cargado un vector V con de componentes.

Ahora bien, el usuario deberá siempre poder determinar la dimensión del vector sobre el que quiera trabajar. Por eso, vamos a leer la dimensión del vector en una variable:

**inicio**

```

    Leer N
    dimensionar V(N)
    desde I = 1 hasta N
        Leer V(I)
    fin desde
fin

```

Así vamos a cargar un vector **SIEMPRE**

### Imprimir un vector

El procedimiento para imprimir un vector es casi exactamente igual al proceso de carga, con la diferencia que en lugar de leer, vamos a imprimir. Para el caso de nuestro vector V de N componentes:

```

    desde I = 1 hasta N
        imprimir V(I)
    fin desde

```

Es bueno volver a señalar que los elementos de un vector son todos del mismo tipo (todos numéricos o todos alfanuméricos).

### Procesando un vector

Ejercicio: Leer un vector de N componentes. Hallar la suma y el promedio de los elementos del vector.

Se pide la suma y el promedio de los elementos. Sabemos que el promedio lo hallaremos dividiendo la suma todos los elementos, sobre la cantidad.

Nuestro vector (al que llamaremos H) va a tener una dimensión D, que será determinada por el usuario.

Siguiendo el esquema que habíamos visto, vamos a tener primeramente un bucle para la carga del vector, otro para el proceso y otro para imprimir el vector.

```

inicio
    suma = 0
    leer
    dimensionar H (D)
    desde I = 1 hasta D
        leer H (I)
    fin desde
    desde I = 1 hasta D
        suma = suma + H(I)
    fin desde
    promedio = suma / D
    desde I = 1 hasta D
        imprimir H (I)
    fin desde
    imprimir ""La suma de los elementos del vector es:", suma
    imprimir "El promedio es:", promedio
fin

```

---

### Arrays

Un array es conjunto finito y ordenado de elementos del mismo tipo (homogéneos). Son ordenados porque siempre se tiene un criterio para identificar el primer, segundo,...n-ésimo elemento. Son homogéneos porque los datos que contiene son todos del mismo tipo (alfanuméricos o numéricos pero no una mezcla de ambos).

El tipo más simple de array es el denominado array unidimensional o vector. Es unidimensional porque solo se necesita un índice para designar la posición de un elemento dentro del array.

Existen datos que están mejor representados en forma de tablas o matrices con dos o más subíndices.

Un tablero de ajedrez es el ejemplo típico de un array de dos dimensiones, ya que se necesitan dos índices para determinar la posición exacta de un elemento dentro del array.

Se pueden definir arrays multidimensionales, es decir de tres, cuatro, cinco.....n-dimensiones; aunque el manejo de arrays con más de tres dimensiones es bastante complicado. En el apartado siguiente vamos a estudiar a los arrays bidimensionales o matrices.

Más ejemplos de vectores en la sección Ejercicios.-

## MATRICES

Un array bidimensional o matriz es un conjunto de datos homogéneos (todos del mismo tipo), cada uno de los cuales debe referenciarse por dos índices. Los índices determinan la posición de una fila y una columna.

En este ejemplo tenemos una matriz de dimensión  $M * N$ , en donde  $M$  es el número de columnas y  $N$  el número de filas. Aquí  $M=5$  y  $N=6$ .

El número total de elementos de la matriz será entonces  $5*6 = 30$ .

De la misma forma que los vectores, una matriz debe tener un nombre. Llamaremos MAT a nuestra matriz ejemplo y determinaremos la posición de algunos de sus elementos. MAT será de tipo alfanumérico.

La matriz MAT está definida con 5 filas y 6 columnas. La notación para el dimensionamiento de una matriz es

NOMBRE (cantidad de filas, cantidad de columnas); luego: MAT(5, 6)

Una vez que la matriz contenga datos (veremos más adelante como cargar una matriz) para referirnos a un elemento debemos conocer en que fila y que columna reside ese elemento, por ejemplo:

MAT (1,1) = "A"  
MAT(3, 5) ="Ñ"  
MAT (4,3)= "OK"  
MAT (5,4)="L"

### **Dimensionamiento y Carga de una matriz**

Así como un vector tiene que ser nombrado y dimensionado antes de ser utilizado, una matriz también. La instrucción para dimensionar una matriz es :

***dimensionar M(5,6)***

La carga de datos se realiza de la misma forma que un vector, por medio de un bucle desde....fin desde; solo que en este caso, vamos a necesitar 2 bucles; uno que recorra las filas y otro las columnas:

***desde fila ↗ 1 hasta 5***

***desde columna ↗ 1 hasta 6***

***leer MAT (fila, columna)***

***fin-desde***

***fin-desde***

en este ejemplo, la variable fila comienza en el valor 1, luego se da inicio al bucle columna que desde 1 hasta 6. El bucle de las columnas debe terminar todo su recorrido para que pueda comenzar el siguiente valor de fila. Los índices van tomando estos valores:

**MAT (1, 1)**

**MAT (1, 2)**

**MAT (1, 3)**

**MAT (1, 4)**

**MAT (1, 5)**

**MAT (1, 6) ----- aquí termina el primer bucle de columnas**

**MAT (2, 1) ----- comienza el segundo bucle para recorrer la segunda fila**

**MAT (2, 2)**

**MAT (2, 3)**

**MAT (2, 4)**

**MAT (2, 5)**

**MAT (2, 6)----- aquí termina el segundo bucle de columnas**

MAT (3, 1) ----- comienza el tercer bucle para recorrer la tercera fila  
 MAT (3, 2)  
 MAT (3, 3)  
 MAT (3, 4)  
 MAT (3, 5)  
 MAT (3, 6)  
 MAT(4, 1) ----- comienza el cuarto bucle para recorrer la cuarta fila  
 MAT(4, 2)  
 MAT(4, 3)  
 MAT(4, 4)  
 MAT(4, 5)  
 MAT(4, 6)  
 MAT(5, 1) ----- comienza el quinto bucle para recorrer la quinta fila  
 MAT(5, 2)  
 MAT(5, 3)  
 MAT(5, 4)  
 MAT(5, 5)  
 MAT(5, 6) ----- Fin de ambos bucles

El recorrido de una matriz se hace, por tanto de esta manera:

Una matriz también puede recorrerse por filas y por cada fila recorrer sus columnas.

Una matriz se imprime utilizando también dos índices:

**inicio**

leer cantfila, cantcolumna  
 dimensionar MAT(cantfila, cantcolumna) ----- se dimensiona  
 desde fila  $\neq 1$  hasta cantfila  
   desde columna  $\neq 1$  hasta cantcolumna  
     leer MAT(fila,columna) ----- lectura  
   fin-desde  
 fin-desde  
 desde fila  $\neq 1$  hasta cantfila  
   desde columna  $\neq 1$  hasta cantcolumna  
     imprimir MAT(fila,columna) ----- impresión  
   fin-desde  
 fin-desde

**fin**

### Procesando una matriz.

Proceso de una matriz se realiza de forma análoga a los ejemplos anteriores. Utilicemos un ejemplo que calcula el promedio de los elementos de una matriz.

**inicio**

leer cantfila, cantcolumna  
 dimensionar M (cantfila, cantcolumna)  
 desde I  $\neq 1$  hasta cantfila  
   desde J  $\neq 1$  hasta cantcolumna  
     leer M(I, J)

```

    fin-desde
  fin-desde
  desde I  $\leftarrow$  1 hasta cantfila
    desde J  $\leftarrow$  1 hasta cantcolumna
      suma  $\leftarrow$  suma + M(I, J)
    fin-desde
  promedio  $\leftarrow$  suma / (cantfila * cantcolumna)
  desde I  $\leftarrow$  1 hasta cantfila
    desde J  $\leftarrow$  1 hasta cantcolumna
      imprimir M(I, J)
    fin-desde
  fin-desde
fin

```

Una matriz que tiene la misma cantidad de filas y de columnas se llama matriz cuadrada.

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 |
| 2 |   |   |   |   |
| 3 |   |   |   |   |
| 4 |   |   |   |   |
| 5 |   |   |   |   |

Esta es una matriz cuadrada de orden 5.

Las matrices cuadradas tienen ciertas características; por ejemplo, los elementos en donde el número de filas es igual al número de columnas se llama diagonal principal (señalados en amarillo): Los elementos de la diagonal principal tienen, entonces, la propiedad de que **fila = columna**

|  |  |  |  |  |
|--|--|--|--|--|
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |

La diagonal principal define así dos áreas bien notorias, una que está por encima y otra por debajo.

La región determinada por los elementos situados sobre la diagonal principal se llama matriz triangular superior que tiene la propiedad de que **fila < columna**.

La matriz triangular inferior es la zona situada debajo de la diagonal principal, cuyos índices cumplen con la propiedad: **fila > columna**.

|  |  |  |  |  |
|--|--|--|--|--|
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |

Elementos de la Matriz triangular superior

Elementos de la Matriz triangular inferior

### Operaciones con Matrices

1- **Suma de matrices:** Si A y B son dos matrices de igual dimensión (MxN), entonces la suma de A y B existe y es igual a una matriz C también de dimensión MxN en donde cada  $C(i, j) = A(i, j) + B(i, j)$ :

Matriz A

|    |    |    |    |
|----|----|----|----|
| 10 | 8  | 3  | 0  |
| 7  | -3 | 33 | 45 |
| 9  | 15 | 71 | 29 |

Matriz B

|    |    |    |    |
|----|----|----|----|
| 1  | 6  | 9  | 69 |
| 14 | 22 | 56 | 7  |
| 3  | 5  | 80 | 1  |

A y B son de igual dimensión, por lo tanto existe una matriz C que es la suma de A y B

|    |    |     |    |
|----|----|-----|----|
| 11 | 14 | 13  | 69 |
| 21 | 19 | 89  | 52 |
| 12 | 20 | 151 | 30 |

2- **Producto Escalar por una matriz:** Si A es una matriz de orden (dimensión)  $M \times N$  y K es un escalar, el producto de  $K \cdot A$  es igual a otra matriz D también de orden  $M \times N$  en donde cada  $D(i, j) = K \cdot A(i, j)$ :

Matriz A:

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  | 6  |
| 7  | 8  | 9  | 10 | 11 | 12 |
| 0  | -1 | -2 | -3 | 21 | 22 |
| 33 | 3  | 0  | 5  | 12 | 4  |

Escalar: 5

Resultado de multiplicar la matriz A por el escalar (numero) 5

Matriz D

|     |    |     |     |     |     |
|-----|----|-----|-----|-----|-----|
| 5   | 10 | 15  | 20  | 15  | 30  |
| 35  | 40 | 45  | 50  | 55  | 60  |
| 0   | -5 | -10 | -15 | 105 | 110 |
| 165 | 15 | 0   | 25  | 60  | 20  |

3- **Traspuesta de una matriz:** Si A es una matriz de orden  $M \times N$ , la traspuesta de A, denotada como  $A^o$ , es otra matriz de orden  $N \times M$  donde cada  $B(i, j) = A(j, i)$ . Una matriz es simétrica si  $A^o = A$ :

Matriz A:

|   |    |    |
|---|----|----|
| 0 | 1  | 2  |
| 3 | 4  | 5  |
| 6 | 7  | 8  |
| 9 | 10 | 11 |

Traspuesta de A,  $A^o$ :

|   |   |   |    |
|---|---|---|----|
| 0 | 3 | 6 | 9  |
| 1 | 4 | 7 | 10 |
| 2 | 5 | 8 | 11 |

## FUNCIONES Y PROCEDIMIENTOS

En general un problema complejo puede ser resuelto de manera más fácil y eficiente si se divide en problemas más pequeños y concentrándonos en cada etapa en la solución de ese "subproblema". Esto implica que el gran problema original será resuelto por medio de varios módulos, cada uno de los cuales se encarga de resolver un subproblema determinado. Esos módulos, se conocen con el nombre de *subalgoritmos*. Un subalgoritmo no es más que un algoritmo que tiene la función de resolver un subproblema.

Los subalgoritmos se escriben sólo una vez, luego es posible hacer referencia a ellos ("llamarlos") desde diferentes puntos de un pseudocódigo. La ventaja obvia es que nos permite reutilización y evita la duplicación de códigos.

Los subalgoritmos son independientes entre si, en el sentido de que se puede escribir y verificar cada módulo en forma separada sin preocuparse por los demás módulos. Por ello, es menos complicado localizar un error y también se puede modificar el código sin tener que tocar o rehacer varias partes del mismo.

Los subalgoritmos pueden ser dos tipos: **Funciones** y **Procedimientos** (también llamadas subrutinas o subprogramas).

Notemos que al utilizar procedimientos y funciones se establece un límite para el alcance de las variables, unas tendrán efecto y valor sólo en el subalgoritmo y otras en el algoritmo principal, también es posible especificar que una variable tenga efecto en el algoritmo principal y todos los subalgoritmos. Este punto lo estudiaremos con más detalle en la sección *Ámbito de variables*.

Los subalgoritmos pueden recibir valores del algoritmo principal (parámetros) , trabajar con ellos y devolver un resultado al algoritmo principal: No existen limitaciones en cuanto a las acciones que pueda ejecutar un subalgoritmo. Un subprograma puede, a su vez, invocar o llamar a otros o a sus propios subprogramas, inclusive puede llamarse a sí mismo (esto se conoce como recursividad).

---

## **Funciones**

Desde el punto de vista matemático, una función es una expresión que toma uno o más valores llamados argumentos y produce un valor que se llama resultado. Este resultado es además, único. Ejemplos de funciones matemáticas son los logaritmos, funciones trigonométricas (seno, coseno, etc).

En el ambiente de programación y diseño de algoritmos, las funciones tienen exactamente el mismo significado. Es decir, se realizan ciertos cálculos con una o más variables de entrada y se produce un único resultado. En programación y diseño de algoritmos, este resultado podrá ser un valor numérico, alfanumérico o lógico. Es decir, una función puede devolver un resultado que puede ser una cadena, un número o un valor de tipo lógico (verdadero o falso). Esto hace que en los lenguajes de programación, debemos especificar de que tipo es una función. Una función será de tipo numérica cuando devuelva un número y será de tipo alfanumérica o string cuando devuelva una cadena. En el caso de las funciones de tipo numérico se tienen subdivisiones que están dadas por los tipos de datos soportados por el lenguaje (integer o entero, simple o single, doble precisión o double, real, etc). O sea que cuando una función numérica devuelva un valor numérico entero (sin decimales) será de tipo entera o integer. Si devuelve un valor decimal será doble o single, dependiendo del grado de exactitud que se desea; sin embargo como esto es propio de lenguajes de programación no se tendrá en cuenta en este tutorial (las funciones serán numéricas cuando devuelvan un dato numérico y de tipo string cuando devuelvan una cadena, sin necesidad de especificar esto previamente en el algoritmo). Puedes consultar al profesor de la materia que te proporcione más detalles sobre los tipos de datos y su aplicación en las funciones.

Tomemos como ejemplo la función matemática  $\sin(x)$ . En este caso la función se llama  $\sin$  (seno) y el argumento o valor que se pasa a la función para que lo procese es  $x$ . Así  $\sin(90^\circ)=1$ . Este valor es además único (por eso se llama función), es decir no existe ningún otro número que la función pueda procesar y devolver 1 más que  $90^\circ$ .

Cuando utilicemos esta función en un pseudocódigo y necesitemos el valor del  $\sin(90^\circ)$ , debemos asignarlo a una variable, así:

valor =  $\sin(90)$

en este caso, la variable valor será = 1, por lo tanto nuestra función es numérica.

Es así como se llaman a las funciones desde un pseudocódigo. Asignándolas siempre a una variable que contendrá el valor devuelto por la función. Si no hacemos esta asignación, la función no podrá ejecutarse ya que no tendrá un "lugar" donde descargar el resultado. Por lo tanto la llamada a una función será siempre:

**variable = funcion (parámetros)**



Ejemplos: La llamada a una función MES que devuelva el nombre del mes, pasándole el valor numérico correspondiente será:  
 nombre\_mes ← MES(2) (esto devolvería "Febrero")  
 La función es de tipo string porque devuelve una cadena como resultado en la variable nombre\_mes.  
 \*\*\*\*\*

Ya sabemos como llamar a una función, ahora veremos como se escribe la función. Como las funciones y procedimientos no se escriben en el algoritmo principal (en programación existen espacios destinados a ellos) todas las funciones y procedimientos que utilice un algoritmo se podrán escribir antes o después del algoritmo principal. Para efectos de este tutorial **las funciones y procedimientos se escribirán siempre al final del algoritmo principal.**

Una función se identifica mediante su nombre. De la misma manera que cuando escribimos un algoritmo comenzamos poniendo: inicio y al final fin, debemos hacer lo mismo para una función. Esto nos dirá donde comienza y donde termina la función. La sintaxis es:

**Función nombre\_funcion (parámetros)**

**<instrucciones>**

**<instrucciones>**

**Fin función**

Todas las funciones devuelven un sólo valor. Siempre debemos indicar a la función mediante una instrucción que devuelva el valor al algoritmo principal (recordemos que la función será llamada desde un algoritmo). Esto se debe hacer en el cuerpo de la función cuando tengamos el resultado. Así que, tomando como ejemplo la función MES, veremos como se escribe el algoritmo principal, como se llama a la función desde el algoritmo principal y cómo se declara la función:

Algoritmo principal

**inicio**

**leer numero\_mes**

**mientras numero\_mes <=0 ó numero\_mes >12**

**imprimir "Debe ingresar un número entre 1 y 12"** > Validación del número entre 1 y 12

**leer numero\_mes**

**fin mientras**

**nombre\_mes ← MES (numero\_mes)**

> Llamada a la función MES

**imprimir "El mes correspondiente es: ", nombre\_mes**

**fin**

**Función MES (valor)**

**Según sea valor**

**caso=1**

**nombre ← "Enero"**

**caso=2**

**nombre ← "Febrero"**

**caso=3**

**nombre ← "Marzo"**

**caso =4**

**nombre ← "Abril"**

**caso=5**

**nombre ← "Mayo"**

**caso=6**

**nombre ← "Junio"**

**caso=7**

**nombre ← "Julio"**

**caso=8**

**nombre ← "Agosto"**

**caso=9**

**nombre ← "Setiembre"**

**caso= 10**

**nombre ← "Octubre"**

**caso=11**

**nombre ← "Noviembre"**

**caso= "12"**

**nombre ← "Diciembre"**

**fin caso**

**MES ← nombre** > **Le decimos a la función que devuelva el resultado al algoritmo principal**

**Fin función**

Debes notar como pasan los valores desde el algoritmo principal a la función. en este caso, cuando se llama a la función

*nombre\_mes*  $\Leftarrow$  *MES (numero\_mes)*

el valor que se pasa a la misma está en la variable *numero\_mes* que toma un valor comprendido entre 1 y 12.

Cuando se llama a la función, este valor debe ser recibido por la misma, en este caso en el cuerpo de la función se coloca entre paréntesis el nombre de la variable que recibirá el valor:

*Función MES (valor)*

Si se pasan varios valores, todos deben ser recibidos en su correspondiente variables.

La función toma el valor pasado desde el algoritmo y lo guarda en la variable *valor* para procesarlo.

Luego de que obtiene un resultado, en este caso el valor de *nombre\_mes*, se le ordena a la función que devuelva ese valor al algoritmo principal:

*MES*  $\Leftarrow$  *nombre*

Esto es siempre así : **nombre\_funcion**  $\Leftarrow$  **resultado**

Es en este punto donde se retorna a la línea siguiente a la que llamó a la función en el algoritmo principal:

*imprimir "El mes correspondiente es: ", nombre\_mes*

Resumiendo. Una función devuelve un sólo valor, para que funcione la función debe recibir uno o varios valores desde el algoritmo principal, realizar el proceso y devolver el resultado. La función se escribe de igual forma que cualquier algoritmo, la diferencia consiste en que en lugar de inicio y fin, ponemos

**Función <nombre\_función>**

**fin\_función**

La llamada a la función se hace con su nombre y el o los valores que le pasamos.

Cuando necesitemos procesar uno o varios valores y ofrecer **UN** resultado, utilizaremos funciones.

\*\*\*\*\*

### Ejemplos de Funciones

A continuación, estudiaremos ejemplos de funciones. De esta forma comprenderemos mejor el su funcionamiento.

**\* Ejemplo1: Escribir una función que devuelva la raíz cuadrada de un número ingresado por teclado.**

Aunque todos los lenguajes de programación tiene instrucciones para calcular una raíz cuadrada, como aquí no estamos escribiendo código, encontraremos la raíz cuadrada de un número elevando a la potencia 1/2. En general, la raíz *x* de un número se obtiene elevando ese número a la potencia 1/*x*. LLamaremos RAIZCUA a la función que vamos a escribir. La función RAIZCUA debe obtener un valor que se pasa desde el algoritmo principal (el número del cual queremos calcular la raíz cuadrada), elevarlo a la potencia 1/2 y luego devolver este valor al algoritmo principal. Recordemos que no podemos permitir el ingreso de números negativos.

**inicio**

*leer numero*

*mientras numero <= 0*

*imprimir "Ingrese un número positivo"*

*leer numero*

*fin-mientras*

*resultado*  $\Leftarrow$  RAIZCUA(*numero*)

*imprimir "La raíz cuadrada es:", resultado*

**fin**

**Función RAIZCUA(valor)**

*raiz*  $\Leftarrow$  *valor* ^ (1/2)

*RAIZCUA*  $\Leftarrow$  *raiz*

**fin-función**

Este algoritmo comienza leyendo el numero, verifica que sea un número positivo con la estructura repetitiva *mientras* y luego hace el llamado a la función RAIZCUA pasándole la variable *numero*. El valor de esa función se recibe en una variable *resultado*.

La función RAIZCUA recibe el numero que el programa le pasa en la variable *valor*, luego eleva ese numero a la potencia 1/2 y lo asigna a la variable *raiz*.

Para que la función devuelva el resultado del proceso al algoritmo principal, se asigna la variable *raiz* a la función, así: **RAIZCUA**  $\Leftarrow$  **raiz** (esto será así para todas las funciones que escribamos).-

Observa que existen variables tanto en el algoritmo principal como en la función. Hablemos de ellas.

\*\*\*\*\*

### Ámbito de las variables

En programación existen dos tipos de variables, las llamadas locales y las variables globales.

**Variables Locales:** Son aquellas que se encuentran dentro de un subprograma (procedimiento o función) y es distinta de las variables que están en el algoritmo principal. El valor se confina al subprograma en el que está declarada.

**Variables Globales:** Son las que se definen o están declaradas en el algoritmo principal y tiene efecto tanto en el algoritmo principal como en cualquiera de sus subprogramas.

Tomando como referencia la función RAIZCUA, las variables globales son: numero y resultado. Y las variables locales son: valor y raíz.

valor y raíz sólo existen en la función RAIZCUA, si en el algoritmo principal tratamos de utilizar estas variables o imprimirlas, no obtendremos nada, ya que para el algoritmo estas variables son locales y desde su punto de vista NO EXISTEN.

numero y resultado son variables globales, es decir que están disponibles en el algoritmo principal y también en la función RAIZCUA.

Una variable local (de un subprograma) no tiene ningún significado en el algoritmo principal y otros subprogramas. Si un subprograma asigna un valor a una de sus variables locales, este valor no es accesible a otros subprogramas, es decir, no pueden utilizar este valor. Las variables globales tienen la ventaja de compartir información de diferentes subprogramas.

En resumen: las variables locales son las que se definen en subprogramas y solo tienen valor dentro de él. Ej.

**Función PROMEDIO(valor1, valor2, valor3)**

**Elpromedio**  $\leftarrow$  **(valor1 + valor2 + valor3) / 3**

**PROMEDIO**  $\leftarrow$  **Elpromedio**

**fin-función**

---

## Procedimientos

Hemos visto que las funciones se utilizan para devolver como resultado un valor Sin embargo, en ocasiones necesitaremos devolver más de un resultado o también ejecutar las mismas líneas de código varias veces en un algoritmo (como por ejemplo una ordenación, etc.) En estas situaciones la función no es apropiada y se utilizarán los procedimientos (también llamados subrutinas).

Un procedimiento es un conjunto de sentencias o instrucciones que realizan una determinada tarea y que pueden ser ejecutados desde más de un punto del programa principal. Un procedimiento tiene una llamada, cuando el procedimiento se ejecuta totalmente, vuelve al punto desde donde fue llamado y se ejecuta la siguiente instrucción.

El procedimiento se escribe como cualquier otro algoritmo, solo existen diferencias en la parte inicial y final. Para nombrar los procedimientos se deben seguir las mismas reglas que para las variables.

Notemos que el objetivo de los procedimientos es ayudar en la modularidad del programa y evitar la repetición de instrucciones ya que estas se pueden escribir en un procedimiento y en lugar de repetirlas, llamar al procedimiento cuantas veces sea necesario.

Desde el programa principal es posible pasar valores (numéricos, alfanuméricos o combinación de ambos) al procedimiento. este utilizará esos valores para realizar un determinado proceso. Los valores que se pasan a un procedimiento (en forma de variables) se llaman parámetros (de igual forma que en las funciones).

### Declaración de un procedimiento

La sintaxis para la declaración de un procedimiento es la siguiente:

**Procedimiento** Nombre\_procedimiento (parámetros)

    <.....acciones...>

    <.....acciones...>

### **Fin Procedimiento**

La llamada a un procedimiento se hace simplemente por su nombre:

**Nombre\_procedimiento(parámetros)**

También es posible que no se pase ningún parámetro al procedimiento, en cuyo caso la llamada se hace así:

**Nombre\_procedimiento()**

Cuando no se pasan parámetros se puede obviar los paréntesis.

**Nombre\_procedimiento**

Podemos utilizar procedimientos, por ejemplo para dibujar recuadros en la pantalla, mostrar mensajes de error, realizar procesos en los que se debe devolver más de un resultado, colocar en un procedimiento las líneas de código que se repiten varias veces en un algoritmo.

Cuando necesitemos devolver más de un valor en un procedimiento, las variables que se devolverán los resultados deben figurar en la lista de parámetros.

**\* Ejemplo: Procedimiento para calcular el cociente y resto de la división entre dos números**  
**inicio**

*leer* numeroA, numeroB

**DIVISION** (numeroA, numeroB, P, Q)

*imprimir* P, Q

**fin**

**Procedimiento DIVISION (dividendo, divisor, cociente, resto)**

**cociente**  $\leftarrow$  dividendo / divisor

**resto**  $\leftarrow$  dividendo - cociente \* divisor

**fin-procedimiento**

En este ejemplo, se pasan los números el dividendo y divisor (numeroA y numeroB respectivamente) y también en los parámetros de llamada al procedimiento deben figurar las variables en las que se devolverán los resultados de cociente y resto (P y Q respectivamente) por eso la llamada es

**DIVISION (numeroA, numeroB, P, Q)**

El procedimiento recibe los valores numeroA en dividendo, numeroB en divisor y se colocan las variables en las que se pasarán al programa principal el cociente y resto. P recibirá el valor de cociente y Q recibirá el valor del resto.

Es decir, que cuando necesitemos devolver más de un valor, los parámetros del procedimiento deben ser los valores que se pasan al procedimiento y luego las variables en las que se recibirán los resultados.

El objetivo de esta sección es ayudar a comprender cómo funcionan las funciones y procedimientos. Puedes pedir a tu profesor más ejemplos de procedimientos y funciones.-

## PREGUNTAS

### 1- Pueden haber problemas para los cuales no existan algoritmos?

Si. Es posible que no exista un algoritmo para resolver un problema. Los problemas realmente complicados a los que se enfrenta el ser humano son aquellos para los cuales no existe algoritmo conocido, bien porque no se haya descubierto aún, o porque definitivamente no exista. Hay problemas para los que se sabe que no existe un algoritmo que los resuelva.

Pongamos como ejemplo el problema del cálculo de la primitiva o integral de una función, Este es un problema para el que no existe algoritmo.

Existen unas reglas que pueden seguirse para obtener la primitiva de ciertas funciones con unas determinadas características, pero no existe un algoritmo genérico que nos diga qué pasos hay que seguir para, partiendo de una función cualquiera, hallar su primitiva. Los matemáticos se basan en ciertas reglas que generalmente funcionan, o que en muchos casos sirven para reducir la complejidad del problema, convirtiendo funciones complicadas en otras más sencillas que sí son capaces de integrar.

Sin embargo la aplicación de estas reglas no garantiza la obtención de una solución en todos los casos.

Las reglas de este tipo, que tratan de orientarnos hacia la solución en problemas no algorítmicos son llamadas heurísticas. Estas reglas son tan concretas como las dadas en los algoritmos. La diferencia reside en el hecho de que nada nos garantiza que la aplicación de una heurística nos lleve un paso más cerca de la solución de un problema. Además, se da la situación de que en muchas ocasiones existe una gran cantidad de heurísticas que son aplicables a un mismo estado del problema, con lo cual la elección de la más adecuada para cada caso puede ser un factor determinante en la obtención de la solución.

Estos son los problemas realmente complejos y que necesitan de verdadera inteligencia y originalidad para su resolución. No hay algoritmos conocidos que nos permitan descubrir teoremas matemáticos, diagnosticar enfermedades, componer música con armonía y gracia o comprender una frase escrita en un lenguaje natural como el español o el inglés, aunque sí haya ciertas heurísticas que la gente aplica, incluso a veces sin darse cuenta. Estas son cosas que las personas hacen sin saber realmente cómo las hacen ni qué pasos siguen en todos los casos, por mucha introspección a la que se sometan.

Estas son las tareas de las que se ocupa la Inteligencia Artificial.

## **2- Existen programas en los cuales se pueden ejecutar y probar los pseudocódigos?**

Un pseudocódigo no puede ser ejecutado directamente, para ello es necesario traducir las instrucciones a sus equivalentes de un lenguaje de programación. Una vez que las instrucciones se han traducido, es posible ejecutar el programa y verificar el resultado.

Para conocer que resultados produce el algoritmo se realiza la llamada "prueba de escritorio", que consiste en realizar el seguimiento de los valores que toman las variables. Esto se hace manualmente, a lápiz y papel.

## **3- Es necesario inicializar las variables numéricas al principio del algoritmo a cero?**

Cuando se trabaja en los pseudocódigos no es obligatorio inicializar las variables numéricas a cero, aunque algunos profesores prefieren requerir esto a los alumnos ya que en algunos lenguajes de programación es necesario. En algunos lenguajes si las variables numéricas no se inicializan a cero, pueden contener valores "basura", lo afecta los cálculos que se puedan realizar en el programa. Por regla general, es conveniente inicializar a cero, todas las variables que aparezcan a ambos lados del símbolo de asignación, como por ejemplo las variables c y suma en el siguiente ejemplo:

$c \leftarrow c + 1$

$\text{suma} \leftarrow \text{suma} + \text{numero}$

Las variables de tipo alfanumérico se pueden inicializar con un valor nulo:

$\text{nombre} = ''$

# EJERCICIOS